

非常抱歉，上一条回复光顾着讲核心逻辑，没有把完整的代码拼装给你。

这里我为你提供**完整、可直接运行的全新 PPO 训练代码**。这套代码完美融合了**“保留 Day 2 作为上下文环境”**和**“仅在 Day 3 进行 240 步决策交互”**的设计。

你可以将以下代码保存为三个 Python 文件，放在同一个目录下直接运行测试。

文件 1：特征工程与数据处理 (data_prep.py)

这个文件负责模拟/处理数据，将 Day 2 的信息压缩为上下文，并生成 Day 3 的 240 步交互状态。

Python

```
import numpy as np
import pandas as pd
import random

def process_single_stock_data(df_day2, df_day3):
    """
    处理单只股票两天的分钟级数据
    df_day2: 第二天的数据 (买入日，包含 240 行)
    df_day3: 第三天的数据 (决策日，包含 240 行)
    """
    # 1. 确定真实买入价 (Day 2 的开盘价)
    buy_price = df_day2['open'].iloc[0]

    # 2. 提取 Day 2 的“上下文总结特征” (静态特征)
    day2_close = df_day2['close'].iloc[-1]
    day2_high = df_day2['high'].max()

    # 昨日实体涨幅 (反映主力做多意愿)
    day2_ret = (day2_close - buy_price) / buy_price
    # 昨日上影线/抛压强度 (反映上方套牢盘压力)
    day2_shadow = (day2_high - day2_close) / buy_price

    # 3. 构建 Day 3 的动态状态矩阵
    states = []
    prices = []
```

```

# 计算 Day 3 的量比 (简单用当前量除以全天均量模拟)
avg_vol = df_day3['volume'].mean() + 1e-8

for i in range(len(df_day3)):
    row = df_day3.iloc[i]
    current_price = row['close']

    # 实时盈亏 (最重要的特征)
    current_pnl = (current_price - buy_price) / buy_price

    # 分钟级波动率
    minute_volatility = (row['high'] - row['low']) / row['open']

    # 分钟级量比
    vol_ratio = row['volume'] / avg_vol

    # 时间流逝比例 (0.0 到 1.0 · 催促模型尽早决策)
    time_decay = i / len(df_day3)

    # 拼接状态向量 (共 6 个基础特征)
    state_vector = [
        current_pnl,      # 1. 实时浮盈
        day2_ret,         # 2. 昨日涨幅 (静态)
        day2_shadow,      # 3. 昨日抛压 (静态)
        minute_volatility, # 4. 当前分钟波动
        vol_ratio,        # 5. 当前分钟量比
        time_decay        # 6. 时间衰减
    ]
    states.append(state_vector)
    prices.append(current_price)

return {
    'states': np.array(states),
    'prices': np.array(prices),
    'buy_price': buy_price,
    'initial_highest_seen': day2_high # 将 Day2 的最高价传入 · 作为计算回撤的基准
}

def generate_mock_data(num_samples=1000):
    """
    生成假数据用于跑通代码
    每条数据包含 480 分钟 (Day 2 = 240, Day 3 = 240)
    """

```

```

trajectories = []
for _ in range(num_samples):
    base_price = random.uniform(10, 50)
    # 随机游走生成 480 步价格
    prices = [base_price]
    for _ in range(479):
        prices.append(prices[-1] * (1 + random.gauss(0, 0.001)))

    df = pd.DataFrame({
        'open': prices,
        'high': np.array(prices) * 1.001,
        'low': np.array(prices) * 0.999,
        'close': prices,
        'volume': np.random.randint(1000, 5000, size=480)
    })

    # 拆分为 Day2 和 Day3
    df_day2 = df.iloc[:240].copy()
    df_day3 = df.iloc[240:].copy()

    traj = process_single_stock_data(df_day2, df_day3)
    trajectories.append(traj)

return trajectories

```

文件 2：强化学习交易环境 (sell_env.py)

这是 PPO 交互的核心。只进行 Day 3 的循环，并在状态中加入“从历史最高点回撤”这一关键动态特征。

Python

```

import gymnasium as gym
from gymnasium import spaces
import numpy as np
import random

class PPOSellTraderEnv(gym.Env):

```

```

def __init__(self, trajectories, fee_rate=0.0015):
    super(PPOSellTraderEnv, self).__init__()

    self.trajectories = trajectories
    self.fee_rate = fee_rate

    # 动作空间 : 0 = 继续持有, 1 = 卖出
    self.action_space = spaces.Discrete(2)

    # 状态空间 : 基础特征 (6个) + 动态回撤特征 (1个) = 7维
    feature_dim = trajectories[0]['states'].shape[1] + 1
    self.observation_space = spaces.Box(low=-np.inf, high=np.inf, shape=(feature_dim,),
dtype=np.float32)

    self.current_traj = None
    self.current_step = 0
    self.max_steps = 0
    self.buy_price = 0
    self.highest_price_seen = 0

    def reset(self, seed=None, options=None):
        super().reset(seed=seed)

        # 随机抽取一只股票的 Day3 数据作为本次回合
        self.current_traj = random.choice(self.trajectories)
        self.current_step = 0
        self.max_steps = len(self.current_traj['prices']) - 1

        self.buy_price = self.current_traj['buy_price']
        # 极其重要 : 最高价初始化为 Day2 的最高价
        self.highest_price_seen = self.current_traj['initial_highest_seen']

    return self._get_obs(), {}

    def _get_obs(self):
        # 基础 6 维特征
        base_features = self.current_traj['states'][self.current_step]
        current_price = self.current_traj['prices'][self.current_step]

        # 更新持仓期最高价
        if current_price > self.highest_price_seen:
            self.highest_price_seen = current_price

```

```

# 动态特征：从已知最高点的回撤幅度 (教模型追踪止盈)
drawdown_from_peak = (current_price - self.highest_price_seen) / self.highest_price_seen

obs = np.append(base_features, drawdown_from_peak)
return obs.astype(np.float32)

def step(self, action):
    current_price = self.current_traj['prices'][self.current_step]
    current_pnl = (current_price - self.buy_price) / self.buy_price

    reward = 0
    terminated = False

    # --- 核心奖励逻辑 ---
    if action == 1: # 卖出！
        terminated = True
        reward = current_pnl - self.fee_rate # 奖励等于扣除手续费后的真实盈亏

    else: # 持有
        self.current_step += 1

        # 1. 强制止损线 (-3%)
        if current_pnl < -0.03:
            terminated = True
            reward = -0.05 # 严重惩罚，逼迫它学会自己尽早止损

        # 2. 到达 Day3 收盘 (240分钟走完)，强制收盘价结算
        elif self.current_step >= self.max_steps:
            terminated = True
            reward = current_pnl - self.fee_rate

        # 3. 继续持有的微小时间惩罚
        else:
            reward = -0.0001

    truncated = False
    return self._get_obs(), reward, terminated, truncated, {"pnl": current_pnl}

```

文件 3：训练启动脚本 (train_trader.py)

使用 stable-baselines3 进行极简且高效的训练。

Python

```
from stable_baselines3 import PPO
from stable_baselines3.common.vec_env import DummyVecEnv # 若要多进程加速可用
SubprocVecEnv
from data_prep import generate_mock_data
from sell_env import PPOSellTraderEnv
import os

def main():
    print("1. 正在准备 PPO 训练轨迹 (包含 Day2 上下文与 Day3 状态)...")
    # 这里生成了 1000 只股票的模拟数据
    # 实战中，请替换为你用 V1.0 分类模型筛选出的 2025年8-9月 真实股票代码数据！
    trajectories = generate_mock_data(num_samples=1000)
    print(f"成功构建 {len(trajectories)} 个股票关卡。")

    print("2. 正在初始化并行环境...")
    # 包装环境 (DummyVecEnv 可以直接运行，对 Windows 友好)
    # 当你准备好大规模训练时，可以改为 SubprocVecEnv 以利用多核 CPU
    env = DummyVecEnv([lambda: PPOSellTraderEnv(trajectories=trajectories)])

    print("3. 初始化 PPO 智能体...")
    model = PPO(
        "MlpPolicy",
        env,
        learning_rate=0.0003,
        n_steps=2048,
        batch_size=64,
        gamma=0.99,
        ent_coef=0.01, # 熵系数，略微提高以增加初期探索
        verbose=1,
        tensorboard_log="./ppo_sell_tensorboard/"
    )

    total_timesteps = 300_000
    print(f"4. 开始训练，总步数 {total_timesteps} 步...")
    # 开始训练！你可以在终端输入 tensorboard --logdir=./ppo_sell_tensorboard/ 观看曲线
    model.learn(total_timesteps=total_timesteps)
```

```

# 5. 保存模型
os.makedirs('models', exist_ok=True)
model.save("models/ppo_sell_expert_v2")
print("\n训练完成！模型已保存至 models/ppo_sell_expert_v2.zip")

# 6. 随机抽一次数据，测试模型实盘表现
print("\n--- 模拟实盘测试一局 ---")
obs = env.reset()
done = False
step_count = 0
while not done:
    # deterministic=True 表示测试时不用随机探索，直接选最高概率动作
    action, _states = model.predict(obs, deterministic=True)
    obs, rewards, done, info = env.step(action)
    step_count += 1

print(f"操作结束！在第 {step_count} 分钟时卖出 (或强平). 最终盈亏 (PnL): {info[0]['pnl']*100:.2f}%")

if __name__ == "__main__":
    main()

```

如何使用这套代码进行测试？

1. 环境准备：确保你的 Python 环境装了：pip install stable-baselines3 gymnasium pandas numpy tensorboard。
2. 冒烟测试：直接运行 python train_trader.py。代码会用随机生成的数据跑 30 万步。你看它能不能顺利跑完并保存模型。
3. 对接真实数据（你的下一步）：在 data_prep.py 中，把你 2025 年 8 月和 9 月的分钟级数据（按我代码里的 df_day2 和 df_day3 格式）传给 process_single_stock_data 函数，替换掉 generate_mock_data，就可以开始真正的闭环验证了！